[redacted]

COP4520

Project 2 Report

Optimized CUDA Techniques for Histogram Computation

Utilizing High-Speed Shared Memory

Shared memory is used to cache frequently referenced atom positions in a block, reducing the number of time-consuming global memory accesses significantly. A block reads a subset of atom positions into shared memory, and all threads within the block can quickly access these positions multiple times during distance computation. This improves memory access efficiency and overall kernel execution time.

Ensuring Safe Concurrent Updates

To prevent race conditions when multiple threads attempt to increment the same histogram bucket, atomic operations are utilized. The atomicAdd function allows multiple threads to safely update the histogram without overwriting each other's changes. This approach prevents complex locking mechanisms while maintaining data integrity, thereby increasing the accuracy and efficiency of the parallel histogram calculation

Optimizing Global Memory Access

Efficient memory access patterns are extremely crucial to maximize GPU performance. When memory accesses are arranged so that all threads of a warp are read from contiguous memory locations, memory transactions are minimized, and throughput is increased. The result is better utilization of the global memory bandwidth, reducing the overhead of uncoalesced memory accesses and achieving better overall performance.

Overview

These optimizations—utilizing shared memory to reduce access latency, using atomic operations to maintain concurrent updates correctly, and structuring memory access for maximum efficiency—act to boost the performance of GPU-based histogram by a considerable amount. By addressing key bottlenecks of memory latency, synchronization, and throughput, these methods enable dramatic speedup over a naive implementation.

Comparative Analysis of Spatial Distance Histogram

Bucket Size: 256 & Block Size: 128

	10000 p	100000 p	500000 p
CPU:	0.92 s	92 s	N/A
GPU:	22 ms	1125 ms	25.9 s